Chapter 12 — Networked Programs (Sockets in Python)

1. Notes (Concept Explanation)

Network Ki Layered Architecture

Network ek **layered architecture** pe kaam karta hai (jese TCP/IP stack). Har layer ka apna kaam hota hai:

- **Link Layer** — Ek hop tak data pohanchane ka kaam karti hai (jese router se router).
- **Internet Layer** — Multiple hops (15-20 tak) ke through packets ko source se destination tak route karti hai.
- **Transport Layer** — Yahan se hum start karte hain. Ye layer ye assume karti hai ke do computers ke darmiyan ek **reliable pipe/connection** ban chuki hai — jese phone call. Hum lower layers (link, internet) ko ignore karte hain kyunke transport layer unko abstract kar deti hai.

> Jab tum browser mein URL open karte ho, request 15-20 hops se guzarti hui server tak jati hai, server data process karke wapis 15-20 hops se hoke tumhare paas aata hai — yehi cheez tumhe webpage ki shakal mein dikhti hai.

Socket Kya Hota Hai?

- Do processes (ek tumhare computer pe, ek server pe) internet ke through connect hote hain — is connection ko **socket** kehte hain.
- Socket ek **bi-directional pipe** hai — dono taraf se data bhej aur receive kar sakte ho (talk + listen).
- Analogy: Ye ek "data phone call" hai application se application ke darmiyan.
  - Client side: Browser (Chrome, Firefox, Internet Explorer)
  - Server side: Web server (IIS, Apache, Tomcat)
- Server generally **hamesha up rehta hai**, aur client jab chahe request bhej sakta hai.

Port Numbers — Extensions Ka Concept

- Ek organization ka ek phone number hota hai, aur us pe "extension" enter karke specific banda milta hai — **ports bilkul yehi kaam karte hain**.
- Server pe multiple services chal rahi hoti hain, har service ek specific **port number** pe "extension" ki tarah sunti hai.
- Convention ke mutabiq standard ports:

Service	Port
Email (SMTP)	25
Telnet (insecure login)	23
HTTP (web)	80
HTTPS (secure web)	443

- Agar URL mein `:number` dikhe (jese `example.com:8080`), iska matlab web server default port (80/443) ki bajaye kisi custom port pe chal raha hai.

Python Mein Socket Banane Ka Tareeqa

```python
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
```

Is code ki breakdown:

1. `import socket` — socket library import karo (Python built-in hai, lekin use karne se pehle import zaroori hai).

2. `socket.socket(...)` — Socket object banata hai. Ye ek **unopened/half file handle** ki tarah hai — outward-looking, lekin abhi tak kisi se connect nahi.
    - `AF_INET` → internet ke across kaam karega
    - `SOCK_STREAM` → **stream socket**, matlab data characters ki continuous series ki tarah aata hai (blocks ki bajaye)
3. `mysock.connect((host, port))` — Ye actual "phone call" hai: given host (`data.pr4e.org`) ke given port (`80`) pe connection banane ki koshish karta hai.
    - Ye line **fail/blow up** ho sakti hai agar us port pe koi service listen nahi kar rahi, network connection nahi, ya server down hai — ye ek legitimate/normal scenario hai jo handle karna padta hai.
4. Is point tak koi data send nahi hua — sirf connection establish hui hai ("phone uthaya gaya"). Agla step decide karna hoga ke kya data send/receive karna hai.

Key Insight

Sirf **3 lines of Python** mein hum us poore internet infrastructure ko use kar rahe hain jo lakhon engineers ne decades mein banaya — yehi wajah hai ke Python itna popular hai (simplicity + abstraction ka power).

---

2. Active Recall Question Bank

1. Transport layer kis cheez ko "assume" kar leti hai jo lower layers handle karti hain?
2. Link layer aur Internet layer mein kya farq hai?
3. Socket ki simple tareeqe se definition do.
4. Socket ko "bi-directional" kyun kaha jata hai?
5. Client aur server side pe kaunse example applications chalte hain?
6. Port numbers ko phone extensions se compare kyun kiya gaya?
7. HTTP aur HTTPS ke default port numbers kya hain?
8. Telnet aur Email (SMTP) ke default ports kya hain?
9. Agar URL mein `:8080` jesa kuch dikhe to iska matlab kya hai?

10. `socket.socket()` call karne se socket "connected" ho jata hai — sahi ya ghalat? Wajah batao.
11. `AF_INET` aur `SOCK_STREAM` parameters ka kya matlab hai?
12. `mysock.connect((host, port))` line fail kyun ho sakti hai?
13. Stream socket ka data doosre socket types se kaise different hota hai?
14. Ek webpage load hone mein packets kitni baar network cross karte hain (roughly)?
15. Kyun kaha gaya ke Python mein sirf 3 lines se hum "half a million engineers" ka kaam use kar rahe hain?

---

3. Answer Key

1. Transport layer ye assume kar leti hai ke ek **reliable pipe/connection** do computers ke darmiyan already ban chuki hai — lower layers (link, internet) ki complexity ko ignore kar deti hai.
2. Link layer ek single hop tak data pohanchati hai; Internet layer multiple hops (~15-20) ke through packets ko end-to-end route karti hai.
3. Socket do processes (ek local, ek remote) ke darmiyan ek connection hai jo internet ke through banti hai, jisse dono talk aur listen kar sakte hain.
4. Kyunke dono taraf se data bhej aur receive kiya ja sakta hai — jese phone call mein dono log bol aur sun sakte hain.
5. Client side: browser (Chrome, Firefox, IE). Server side: web server software (IIS, Apache, Tomcat).
6. Kyunke ek organization ka ek phone number hota hai aur extension se specific banda milta hai — similarly server ka ek address hota hai aur port se specific service/process milta hai.
7. HTTP → 80, HTTPS → 443.
8. Telnet → 23, Email (SMTP) → 25.
9. Iska matlab hai web server default port (80 ya 443) ki bajaye kisi custom/non-standard port pe run ho raha hai.

10. **Ghalat.** `socket.socket()` sirf ek unopened/unconnected socket object banata hai (jese half file handle) — actual connection `connect()` call se banti hai.
11. `AF_INET` batata hai ke socket internet address family use karega; `SOCK_STREAM` batata hai ke ye ek stream-type socket hai jahan data continuous character stream ki tarah aata hai, block-wise nahi.
12. Kyunke ho sakta hai us specified port pe koi service listen na kar rahi ho, network connection na ho, ya poora server hi down ho — ye sab legitimate real-world scenarios hain.
13. Stream socket data ko ek continuous series of characters ki tarah deta hai (ek ke baad ek), jabke doosre types (jese datagram) data blocks/chunks mein deal karte hain aur handle karna zyada mushkil hota hai.
14. Roughly 15-20 baar (request jaate hue aur response wapis aate hue dono taraf).
15. Kyunke Python ki socket library ne underlying complex network protocols (jo decades mein lakhon engineers ne design kiye) ko itna abstract kar diya hai ke sirf `import socket`, `socket.socket()`, aur `.connect()` — 3 lines se ek working internet connection ban jata hai.

Part 2 — Application Protocol, HTTP, aur Request-Response Cycle

1. Notes (Concept Explanation)

Application Protocol Kya Hota Hai?

- Socket sirf ek **connection/endpoint** banata hai — `connect()` se hum kisi far host (jese `www.py4e.org`) ke kisi port (jese 80) tak pohanch jate hain.
- Lekin sirf connect hone se kaam nahi chalta — ab ye decide karna hota hai ke us connection pe **kya bhejna hai aur kya expect karna hai**. Yehi rules **application protocol** kehlate hain.
- Analogy: Phone call connect hone ke baad "Hello" kehna ek protocol hai — dono taraf ko pata hona chahiye ke pehle kaun bolega aur kya bolega.

- Application protocol ka kaam: decide karna ke **kaun pehle "baat" karega, kya kahega, aur agar doosri taraf jawab na de to kya hoga.**

HTTP — Sab Se Common Application Protocol

- Is course mein hum jo protocol use karenge wo hai **HTTP (HyperText Transfer Protocol)** — simple aur widely-used hai.
- Lekin HTTP akela protocol nahi — mail, file transfer, remote login jese kaam ke liye alag-alag application protocols hote hain. Sab ki base **socket** hoti hai, lekin har ek ke apne "rules of the road" hain (jese traffic mein left ya right side drive karna — koi bhi convention chale, bas sab agree hona chahiye).

URL — Web Ka Genius Idea

- URL (Uniform Resource Locator) mein 3 cheezein encode hoti hain:
  1. **Protocol** (jese http, ftp)
  2. **Host** (kis server se baat karni hai)
  3. **Document/resource** (kya retrieve karna hai)
- Ye 1990s mein bohat revolutionary tha, kyunke pehle har protocol (FTP, Telnet, SMTP) ke liye alag program use karna padta tha aur manually host se connect karna padta tha.
- HTTP ko **Tim Berners-Lee aur Robert Cailliau** ne **1989-1990** mein **CERN** mein invent kiya tha.

Request-Response Cycle (Click Se Page Load Tak)

Jab user kisi webpage pe link pe click karta hai, ye sequence hoti hai:

1. **Browser (client app)** link ka click detect karta hai.
2. Browser server (jese `drchuck.com`) ke **Port 80** pe connection banata hai.
3. Browser ek precisely-specified **request** bhejta hai (standard format follow karke).
4. **Server** apna kaam karta hai — files read karta hai, code run karta hai, aur ek response construct karta hai.

5. Server response wapis bhejta hai — usually **HTML** (HyperText Markup Language) format mein.
    - Yaad rahe: **HTTP = protocol** (baat karne ka tareeqa), **HTML = format** (document ka structure) — dono different cheezein hain.
6. Browser HTML/CSS/JS ko parse karta hai aur ek "pretty" webpage render karta hai.
7. Agar us page mein aage koi link ho, to click karne pe ye poora cycle **dobara** hota hai.

Ye poora process **Request-Response Cycle** kehlata hai.

Internet Standards aur RFC

- HTTP jese protocols standard documents pe based hain jo **IETF (Internet Engineering Task Force)** publish karti hai.
- In documents ko **RFC (Request For Comments)** kaha jata hai — naam thora ironic hai kyunke decades purane established protocols abhi bhi "RFC" (comments ke liye request) kehlate hain, matlab community hamesha improvement ke liye open rehti hai.
- Example: **RFC 2616** — HTTP/1.1 ko define karta hai. Isme exact rules likhe hote hain, jese request ka format: `METHOD [space] URI [space] HTTP-VERSION [carriage-return][line-feed]`.
    - Yahan spacing bhi strict hai — do spaces use karoge to protocol fail ho jayega.

Telnet Se Manually HTTP Request Bhejna

- **Telnet** ek purana program hai jo kisi host ke kisi port pe raw socket connection khol deta hai — jo bhi type karo wo seedha server ko bhej diya jata hai.
- Aajkal login ke liye use nahi hota kyunke ye data ko **encrypt nahi karta** (insecure), lekin sockets samajhne ke liye acha demonstration tool hai.
- Command: `telnet www.py4e.org 80` — is se manually port 80 pe connect hoke, hath se HTTP request type karke server se response le sakte ho.

- Important: Request **fast type karni padti hai** aur end mein **do baar Enter** (ek blank line) dena zaroori hai — warna server "human" samajh kar request ignore/reject kar sakta hai (kyunke servers programs se baat karne ke liye designed hote hain, slow-typing humans se nahi).
- Response mein pehle **headers** (document ke baare mein metadata) aate hain, phir ek blank line, phir **actual document**, aur phir connection close ho jata hai.

Command Line Aur Hacking Ka Connection

- Command line se protocols ko directly bolna seekhna important hai — yehi tareeqa hai jaise **real hackers** kaam karte hain: wo GUI ki "pretty clicky buttons" ke peeche jo raw protocol chal raha hota hai usay directly samajhte aur use karte hain.
- Secure servers ab pehle jitna asaan hack karna nahi (challenge-response mechanisms ki wajah se), lekin underlying protocols samajhna developer/hacker dono ke liye foundational skill hai.

Aage Kya?

Agla step: Yehi HTTP request jo humne manually telnet se type ki, wo ab **Python code se automatically bhejni hai** — yani ek simple **Python web browser** likhna.

---

2. Active Recall Question Bank (Part 2)

1. Application protocol kis cheez ko decide karta hai (socket connect hone ke baad)?
2. HTTP full form aur iska basic role kya hai?
3. URL mein kaunsi 3 cheezein encode hoti hain?
4. URL invent hone se pehle log different protocols (FTP, Telnet, SMTP) kaise use karte the?
5. HTTP ko kis ne aur kahan invent kiya, aur kab?
6. HTTP aur HTML mein basic farq kya hai?
7. Request-Response cycle ke 7 steps summarize karo (click se render tak).
8. IETF aur RFC kya hain?
9. RFC 2616 kis cheez ko define karta hai?

10. HTTP request ka basic format (method, URI, version) kya hota hai, aur spacing kyun important hai?
11. Telnet kya hai aur is se server se manually kaise baat ki ja sakti hai?
12. Telnet se HTTP request bhejte waqt kya dhyan rakhna zaroori hai (2 cheezein)?
13. Server response mein kya-kya include hota hai (order mein)?
14. Telnet ko login ke liye kyun avoid kiya jata hai aajkal?
15. Command line samajhna hacking/security perspective se kyun important hai?

---

3. Answer Key (Part 2)

1. Application protocol decide karta hai ke connection pe **kya data bheja jayega aur kya receive expect kiya jayega** — yani "rules of the road" (kaun pehle bolega, kya bolega, no-response pe kya hoga).
2. **HyperText Transfer Protocol** — ye web documents (pages) ko retrieve karne ka standard, simple protocol hai.
3. **Protocol** (jese http), **Host** (jese www.py4e.org), aur **Document/resource** (jo retrieve karna hai).
4. Har protocol (FTP, Telnet, SMTP) ke liye alag program use karna padta tha, aur manually us host se sahi tareeqe se connect karna padta tha — koi unified system nahi tha.
5. **Tim Berners-Lee aur Robert Cailliau** ne, **CERN** mein, **1989-1990** mein invent kiya.
6. **HTTP** communication ka **protocol** hai (baat karne ka tareeqa), jabke **HTML** us document ka **format/structure** hai jo response mein milta hai.
7. (1) Browser click detect karta hai → (2) Server ke port 80 pe connect karta hai → (3) Request bhejta hai → (4) Server process karke response banata hai → (5) Server HTML response bhejta hai → (6) Browser HTML/CSS/JS parse karke render karta hai → (7) Agla link click hone pe cycle repeat hoti hai.
8. **IETF (Internet Engineering Task Force)** wo group hai jo internet standards banata hai; **RFC (Request For Comments)** un standards ke official documents hain.

9. RFC 2616 **HTTP version 1.1** ko define karta hai — request/response ka exact format aur rules.
10. Format: `METHOD [space] URI [space] HTTP-VERSION [CRLF]`. Spacing strict hai kyunke ye protocol specification ka hissa hai — extra ya missing space se request fail ho sakti hai (parser confuse ho jata hai).
11. Telnet ek program hai jo kisi host ke kisi port pe raw socket khol deta hai; jo bhi keyboard se type karo wo seedha us server ko bhej diya jata hai — is tarah manually HTTP request type karke server se raw response dekha ja sakta hai.
12. (1) Request **fast type** karni chahiye (warna server human samajh kar reject kar sakta hai), (2) End mein **do baar Enter** (blank line) dena zaroori hai.
13. Pehle **headers** (metadata about the document), phir ek **blank line**, phir **actual document/content**, phir connection **close** ho jata hai.
14. Kyunke Telnet data ko **encrypt nahi karta** — isliye ye insecure hai aur credentials/data plaintext mein interceptable hote hain.
15. Kyunke real hackers GUI ki "pretty" layer ke peeche jo raw protocol communication ho rahi hoti hai usay directly samajhte aur manipulate karte hain — command line se protocols ko directly bolna unhe developers se better system understanding deta hai.

Part 3 — Python Mein Web Browser Likhna

1. Notes (Concept Explanation)

Poora Flow — Socket Se Le Kar Data Print Tak

Ab hum wahi kaam jo pehle Telnet se **hath se** kiya tha, usay **Python code** se automate karenge. Poora program bas ~**10 lines** ka hai.

```python
```

```python
import socket

mysock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
mysock.connect(('data.pr4e.org', 80))
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)

while True:
    data = mysock.recv(512)
    if len(data) < 1:
        break
    print(data.decode())

mysock.close()
```

Step-by-Step Breakdown

1. **Socket banana aur connect karna** (jo pichle sections mein cover ho chuka):
   - `socket.socket(socket.AF_INET, socket.SOCK_STREAM)` → stream-based socket banata hai jo internet ke across chal sakta hai.
   - `mysock.connect(('data.pr4e.org', 80))` → server ko "phone call" karta hai. Agar us port pe koi software listen nahi kar raha to ye **blow up (fail)** ho jayega.
2. **"Kaun pehle bolega?" — Application Protocol Ka Decision**
   - Connection banne ke baad, web server **kuch nahi bolta** — wo chup rehta hai aur client (browser) ka wait karta hai.
   - Isliye **hum (client) pehle baat karte hain** — ye HTTP protocol ka rule hai.
3. **GET Request Banana Aur Bhejna**
   - Request ek **string** ki tarah likhi jati hai: `GET <path> HTTP/1.0` + do newlines (blank line zaroori hai — jese pichle section mein telnet se manually type karte waqt tha).

- `.encode()` — Ye string ko **bytes** mein convert karta hai, kyunke network pe data bytes ki form mein travel karta hai (raw text ki tarah nahi). Isay **"encoding"** kehte hain.
- `mysock.send(cmd)` — Ye encoded request ko socket ke through actual network pe bhej deta hai.

4. **Server Ka Response**
    - Server request receive karke apna kaam karta hai (file read, code run, etc.) aur phir data wapis bhejna start karta hai.
5. **Data Receive Karne Ka Loop**
    - `mysock.recv(512)` — Ek time mein **512 characters/bytes** receive karta hai (chunks mein, pura data ek sath nahi aata).
    - Loop tab tak chalta hai jab tak socket band na ho jaye.
    - **Agar `recv()` se 0 bytes milein (`len(data) < 1`), iska matlab stream/connection close ho gaya hai** — ye end-of-data ka signal hai (jese Telnet session mein "connection closed" dikhta tha).
    - Agar abhi connection open hai lekin data slow aa raha hai, to `recv()` **wait** karega (pause). Slow network pe ye pause noticeable hota hai; fast network pe itni jaldi hota hai ke pata hi nahi chalta.
6. **Decode Karna Zaroori Kyun Hai?**
    - Data jo outside world (network) se aata hai, wo **bytes** ki form mein hota hai — usay print karne se pehle `.decode()` se wapis **string/text** mein convert karna padta hai.
7. **Loop Khatam, Socket Close**
    - Jab data khatam ho jaye (0 bytes mil jayein), loop `break` ho jata hai aur `mysock.close()` se connection band kar di jati hai.

Output Kaise Dikhta Hai?

- Bilkul waisa hi jaisa Telnet se manually karne pe dikhta tha:

1. Pehle **headers** (metadata — jese ye batata hai ke response mein aage kya type ka content aa raha hai).
2. Ek **blank line** — ye signal hai ke headers khatam, ab actual data/content start ho raha hai.
3. Phir **actual content** (is example mein `romeo.txt` file ka text).

Important Caveat — Sirf Text Ke Liye Kaam Karta Hai (Is Form Mein)

- Ye simple program **sirf text data** (jese `.txt` file) ke liye theek se kaam karta hai.
- Agar isi code se koi **image (jpeg/png)** retrieve karne ki koshish ki jaye, to output **gibberish/garbage characters** dikhega — kyunke binary data (images) ko simple `.decode()` se readable text mein convert nahi kiya ja sakta. (Isay properly handle karne ka tareeqa aage discuss hoga.)

Next Topic Ka Preview

Aage ye dekha jayega ke jab characters computer ke **andar (strings)** se **socket ke through bahar (network)** jate hain aur wapis aate hain, to unke saath **encoding/decoding** level pe exactly kya hota hai.

---

2. Active Recall Question Bank (Part 3)

1. Python mein poora simple web browser kitni lines ka ban jata hai?
2. Connection banne ke baad web server pehle kuch bolta hai ya client? Kyun?
3. GET request bhejne se pehle `.encode()` kyun call kiya jata hai?
4. `mysock.send(cmd)` ka kaam kya hai?
5. `mysock.recv(512)` mein `512` ka kya matlab hai?
6. Loop mein `if len(data) < 1: break` line ka kya matlab/purpose hai?
7. Agar network slow ho to `recv()` call ka behavior kaisa hoga?
8. `.decode()` ki zaroorat kyun parti hai response data pe?
9. Server response mein headers aur actual data ke darmiyan kaunsi cheez separator ka kaam karti hai?

10. Ye simple program agar image (jpeg/png) retrieve kare to kya hoga aur kyun?
11. GET request string mein do newlines (`\r\n\r\n`) kyun zaroori hain?
12. Socket connect() call kab "blow up" (fail) ho sakti hai?
13. Poora request-response cycle is program mein kin steps mein hota hai (short list)?

---

3. Answer Key (Part 3)

1. Sirf ~**10 lines** ka.
2. **Client (browser)** pehle bolta hai — web servers connection ke baad chup rehte hain aur client ka wait karte hain; ye HTTP application protocol ka established rule hai.
3. Kyunke network pe data **bytes** ki form mein bhejna hota hai, jabke Python string normally text/unicode hoti hai — `.encode()` string ko bytes mein convert karta hai taake wo network-transmittable ho.
4. Ye encoded request (bytes) ko socket ke through actual network par server tak bhejta hai.
5. `512` batata hai ke ek `recv()` call mein **maximum kitne bytes/characters ek dafa mein receive kiye jayein** — data chunks mein aata hai, pura ek sath nahi.
6. Ye check karta hai ke agar receive hui data ki length 0 ho (matlab kuch data nahi mila), to iska matlab connection/stream **close ho chuka hai** — is condition pe loop `break` ho kar khatam ho jata hai.
7. `recv()` **wait/pause** karega jab tak naya data na aaye — slow network pe ye pause visible hota hai (agar print statements ho), fast network pe itni jaldi ho jata hai ke notice nahi hota.
8. Kyunke network se aane wala data **raw bytes** ki form mein hota hai — usay human-readable text/string mein print karne se pehle `.decode()` se convert karna zaroori hai.
9. Ek **blank line** — ye headers (metadata) aur actual content ke darmiyan separator ka kaam karti hai.

10. Output **gibberish/garbage** dikhega, kyunke image binary data hai aur `.decode()` (jo text ke liye hai) us binary data ko sahi tarah text mein convert nahi kar sakta.
11. Ek newline HTTP request line khatam karne ke liye, aur doosri (blank line) ye signal dene ke liye ke request headers/body khatam ho gaye hain aur server ab response bhej sakta hai — ye HTTP protocol ki requirement hai.
12. Jab specified host/port pe koi service listen nahi kar rahi ho, ya network down ho, ya server hi down ho — tab `connect()` fail (blow up) ho sakti hai.
13. (1) Socket banao, (2) Server se connect karo, (3) GET request encode karke bhejo, (4) Loop mein chunks (512 bytes) receive karo jab tak connection close na ho, (5) Har chunk decode karke print karo, (6) Socket close karo.

Part 4 — Character Sets: ASCII, Unicode, UTF-8, aur Encode/Decode

1. Notes (Concept Explanation)

ASCII — Purana Simple Character Set

- 1950s-60s mein log ek hi computer se khush the, internal representation ki fikar nahi thi — bas keyboard pe jo type karo wahi printer pe nikal aaye.
- 70s-80s mein **interoperability** (different systems ka aapas mein sahi tarah data samajhna) ki zaroorat hui, to character set ko **standardize** kiya gaya — ye standard **ASCII** kehlaya.
- ASCII bunyadi taur pe **Latin/Western character set** hai (US, Europe use karte hain) — isme Spanish, French tak ke special characters nahi hain, chaye ye Western languages hi hon.
- ASCII mein **128 possible values** hain:
    - Kuch **non-printing characters** (whitespace, control characters)
    - Kuch **printing characters** (numbers, uppercase letters, lowercase letters, symbols)

- Har character ek number se **map** hota hai — e.g., lowercase `a` → decimal **97** (hex mein `0x61`, octal mein `141`).
- Ye numbers **8 bits = 1 byte** mein store hote hain — isi wajah se disk storage "bytes/megabytes/gigabytes" mein measure hoti hai.
- **Important detail:** Uppercase letters lowercase se numerically **chhoti** values pe hain — isi liye Latin-based sorting mein uppercase letters lowercase se pehle aate hain (jese `"Zebra" < "apple"` alphabetically nahi lagta lekin ASCII sort mein aisa hi hota hai).

Python Mein `ord()` Function

- `ord()` kisi character ki **numeric/ordinal value** batata hai.
- Example: `ord('H')`, `ord('e')`, `ord('\n')` — newline ki ordinal value **10** hoti hai.
- Ye sorting behavior ko bhi explain karta hai — jaise lowercase `e` ki value uppercase `H` se zyada hai, isliye simple numeric sort mein `e` `H` ke baad aayega.

ASCII Ki Limitation → Unicode Ki Zaroorat

- Modern duniya mein sirf ASCII (Latin characters) kaafi nahi — har language/character set (Chinese, Arabic, Urdu, emojis, etc.) ko support karna zaroori hai.
- Is wajah se **Unicode** banaya gaya — ismein **billions of possible characters** ke liye space hai, har language/character set ke liye.
- Unicode itna bada hai ke chhoti chhoti character variations ko bhi apni alag jagah mil jati hai.

Unicode Ko Network Pe Bhejne Ka Masla — Encoding Formats

- Agar hum full Unicode (jese **UTF-32**, jo **4 bytes per character** hai) directly network pe bhejein, to data **4 guna bara** ho jayega — bohat inefficient.
- Isliye Unicode ko **compress** karne ke tareeqe bane:

Encoding	Bytes per character	Notes
ASCII	1 byte (fixed)	Sirf 128 characters, Latin/Western
UTF-16	2 bytes (roughly)	Unicode ka subset, kuch countries mein used
UTF-32	4 bytes (fixed)	Full Unicode representation
UTF-8	**1 to 4 bytes (dynamic)**	**Best practice** — internet standard

- **UTF-8 sab se important hai** kyunke:
  - Iski length **dynamic** hai (1-4 bytes, jitni zaroorat ho).
  - Agar character sirf 1 byte mein fit ho jaye, to wo **exactly ASCII ke barabar** hota hai — matlab **har ASCII file automatically ek valid UTF-8 file bhi hai** (backward compatible).
  - UTF-8, UTF-32 ki tarah **sab kuch represent kar sakta hai**, bas compressed form mein.
- 2000s se 2014 tak internet pe documents overwhelmingly **UTF-8** ki taraf shift hui hain — aaj UTF-8 hi dominant/recommended standard hai.

Python Mein Strings Vs Bytes — Python 2 Vs Python 3 Ka Farq

- **Python 2:**
  - Normal `string` type = **byte string** (ASCII/Latin based).
  - Unicode ke liye ek **alag object type** tha.
  - Yani: *Bytes aur Strings same the, Unicode "weird"/separate tha.*
- **Python 3:**
  - Normal `string` type = **Unicode** (built-in).
  - Bytes ab ek **alag type** hai (`bytes` object).
  - Yani: *Strings aur Unicode same hain, Bytes "weird"/separate hain.*
- Is course mein hum **Python 3** use kar rahe hain, isliye:

- Python ke andar (variables, files padhna/likhna) sab kuch **Unicode string** hota hai — normally isay explicitly handle nahi karna parta.
- Lekin **sockets/network** ke through data bahar (outside world) jata hai, jahan sirf **bytes** samajhi jati hain — is wajah se yahan encode/decode ka khayal rakhna zaroori ho jata hai.

Byte Object Vs String Object

- `bytes` object ek **array of bytes** hai — e.g., `A`, `B`, `C` = **3 bytes**.
- Ek `string` (Unicode) mein wahi 3 characters, encoding ke hisaab se, **3 se 12 bytes tak** ho sakte hain (depend karta hai character kis language ka hai aur UTF-8 mein kitni jagah leta hai).
- `bytes` object `len()` jese functions ke sath **string ki tarah behave** karta hai, lekin internally ek byte per position hota hai.

Encode Aur Decode — Konsa Kab Use Karna Hai

- `.encode()` → Unicode **string ko bytes mein** convert karta hai (**bhejne se pehle**, socket se bahar bhejte waqt).
  - Default UTF-8 use hota hai (chaho to parameter se doosri encoding specify kar sakte ho, lekin practically almost hamesha UTF-8 hi use hota hai).
- `.decode()` → **bytes ko wapis Unicode string mein** convert karta hai (**receive karne ke baad**, print/use karne se pehle).

Simple Rule:

> String → `.encode()` → Bytes (bhejne ke liye) Bytes → `.decode()` → String (use/print karne ke liye)

Code Mein Practical Example

```
python
```

```
cmd = 'GET http://data.pr4e.org/romeo.txt HTTP/1.0\r\n\r\n'.encode()
mysock.send(cmd)

data = mysock.recv(512)
print(data.decode())
```

- `cmd` shuru mein ek **Unicode string** hai (chahe usme koi special/non-Latin character na bhi ho, phir bhi Python 3 mein wo Unicode hi hoti hai).
- `.encode()` call hone ke baad `cmd` ek **bytes object** ban jata hai — ab isay `send()` se network pe bheja ja sakta hai.
- Jab data wapis aata hai, `mysock.recv(512)` se jo milta hai wo **bytes** hota hai, use print karne se pehle `.decode()` zaroori hai taake wo wapis readable string bane.

Practical Reality

- **95-98% of the time**, internet pe data **UTF-8** (jo ASCII ko bhi cover karta hai) hi hota hai — is wajah se `decode()`/`encode()` bina extra parameter ke (default UTF-8) zyadatar kaam kar jata hai.
- Lekin kabhi kabhi APIs se aisi bytes mil sakti hain jo kisi aur encoding (jese UTF-16) mein hon — tab humein manually specify karna parta hai.

Aage Kya?

Ab jab socket-level encode/decode ka concept clear ho gaya hai, agla step hai **`urllib`** — Python ki ek library jo saara socket-level kaam (connect, send, recv, encode/decode) khud handle kar leti hai, taake humein manually sockets ke sath deal na karna pade.

---

2. Active Recall Question Bank (Part 4)

1. ASCII character set kitne possible values support karta hai, aur ye kis "culture/language group" ke liye designed tha?
2. Lowercase `a` ki decimal ordinal value kya hai?
3. Uppercase letters, lowercase letters se sorting mein pehle kyun aate hain?

4. Python mein `ord()` function ka kaam kya hai?
5. Newline character (`\n`) ki ordinal value kya hai?
6. Unicode kyun banaya gaya (ASCII ki konsi limitation solve karne ke liye)?
7. UTF-32 ko directly network pe bhejna kyun inefficient hai?
8. UTF-8 ko "best practice for internet" kyun kaha jata hai (2 reasons)?
9. UTF-8, ASCII se backward compatible kaise hai?
10. ASCII, UTF-16, aur UTF-32 mein se kaunsa "fixed length" hai aur kaunsa "dynamic length"?
11. Python 2 aur Python 3 mein `string` type ka basic difference kya hai?
12. Python 3 mein `bytes` object kya represent karta hai?
13. Ek 3-character Unicode string, bytes mein kitni length le sakti hai (min-max)?
14. `.encode()` aur `.decode()` mein basic farq kya hai — konsa kab use hota hai?
15. Socket code mein `cmd.encode()` ke baad `cmd` ka type kya ho jata hai?
16. `mysock.recv(512)` se jo data milta hai uska type kya hota hai, aur usay print karne se pehle kya karna zaroori hai?
17. Roughly kitne percent internet data aaj UTF-8 (ya uske compatible ASCII) mein hota hai?
18. Sockets ke sath kaam karte waqt encode/decode ka khayal kyun rakhna parta hai, jabke normal Python program (files read/write) mein aksar iski zaroorat nahi parti?

---

3. Answer Key (Part 4)

1. **128** possible values; ye mainly **Western/Latin** languages (US, Europe) ke liye design hua tha.
2. **97** (decimal); hex mein `0x61`, octal mein `141`.
3. Kyunke ASCII mapping mein uppercase letters ki numeric values lowercase letters se **kam (chhoti)** hain, aur default sorting numeric value ke hisaab se hoti hai.
4. `ord()` kisi character ki **numeric/ordinal (integer) value** return karta hai uske character set mapping ke mutabiq.

5. **10.**
6. Kyunke ASCII sirf Latin/Western characters cover karta tha — Spanish, French, Chinese, Arabic, Urdu, emojis jese har language/symbol ke liye jagah nahi thi; Unicode ye limitation solve karta hai billions of characters ki capacity de kar.
7. Kyunke UTF-32 **fixed 4 bytes per character** use karta hai — is se data size **4 guna bara** ho jata hai, jo network transfer ke liye bohat inefficient hai.
8. (1) Iski length **dynamic** hai (1-4 bytes, zaroorat ke mutabiq — efficient), (2) Ye **ASCII se backward compatible** hai (1-byte case mein).
9. Agar UTF-8 mein encode hone wala character sirf **1 byte** mein fit ho jaye, to wo bytes bilkul ASCII ke encoding jaisi hoti hain — isliye har valid ASCII file automatically ek valid UTF-8 file bhi hoti hai.
10. **Fixed length:** ASCII (1 byte) aur UTF-32 (4 bytes). **Dynamic length:** UTF-8 (1 se 4 bytes).
11. **Python 2:** `string` = byte string (ASCII-based); Unicode alag type tha. **Python 3:** `string` = Unicode (default); bytes ek alag (`bytes`) type hai.
12. `bytes` object ek **array of raw bytes** represent karta hai — network se receive hui ya bhejne wali raw data, jo abhi tak Unicode string mein decode nahi hui.
13. **3 se 12 bytes** tak (depend karta hai characters kis type ke hain — simple ASCII-range characters kam bytes lenge, complex/non-Latin characters zyada).
14. `.encode()` — **string → bytes** (bhejne se pehle use hota hai); `.decode()` — **bytes → string** (receive karne ke baad, use/print karne se pehle use hota hai).
15. `cmd` ek **bytes object** ban jata hai (ab wo Unicode string nahi rehta).
16. Data ka type **bytes** hota hai; print karne se pehle `.decode()` call karna zaroori hai taake wo wapis readable **string** ban jaye.
17. Roughly **95-98%**.
18. Kyunke sockets ke zariye data **Python ke bahar, network/outside world** ke sath exchange hota hai jahan sirf **bytes** samajhi jati hain — jabke Python ke andar (variables, file I/O mein) strings already **Unicode** hoti hain aur Python khud is

conversion ko largely automatically/transparently handle kar leta hai, isliye wahan explicit encode/decode ka zikar nahi hota.

Part 5 — urllib Library (Web Browser Sirf 4 Lines Mein)

1. Notes (Concept Explanation)

urllib Kya Hai Aur Kyun?

- Pichle section mein humne **socket** use karke ek web browser **10 lines** mein likha tha (connect karna, GET request encode/bhejna, loop mein receive karna, decode karna, close karna).
- Chunke ye poora pattern (connection banana, GET request bhejna, blank line add karna, headers alag karna, data retrieve karna) itna **common** hai, isay ek library mein daal diya gaya — is library ka naam `urllib` hai.
- `urllib` isi socket-level complexity ko **abstract** kar deti hai, taake developer ko manually socket handle na karna pade.

Basic Usage — `urlopen()`

```python
import urllib.request

fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')
for line in fhand:
    print(line.decode().strip())
```

- `import urllib.request` — socket ki jagah ab `urllib` library import hoti hai (specifically `request` module).
- `urllib.request.urlopen(url)` — Function call jisay ek **URL string** diya jata hai.
  - Ye function internally: **connection banata hai, GET request khud "encode" karke bana leta hai, use bhejta hai, aur headers ko retrieve karke apne paas**

"keep" kar leta hai (headers ko baad mein alag se access kiya ja sakta hai, lekin default mein wo directly nahi dikhte).

- Ye sab wahi steps hain jo humne pichle section mein manually socket se kiye the — `urlopen()` unhe **automatically** kar deta hai.

- **Return value:** `urlopen()` ek aisa object return karta hai jo **file handle jaisa behave karta hai** — isi wajah se isay `for line in fhand:` jese **for loop** mein directly use kiya ja sakta hai, bilkul waise jaise local file ko read karte hain.

Important Detail — Decode Abhi Bhi Zaroori Hai

- Loop se jo `line` milti hai, wo **bytes** hoti hai — `urllib` automatically decode **nahi** karta.
- Isliye `line.decode()` manually call karna zaroori hai (kyunke encoding/character set situation-dependent ho sakti hai, isliye Python ye decision developer pe chorta hai).
- `.strip()` — extra whitespace/newline characters hata deta hai.

Headers Kahan Gaye?

- Is basic loop mein **headers dikhte nahi** — kyunke `urlopen()` unhe internally "consume/eat" kar ke apne paas store kar leta hai.
- Agar headers chahiye hon, to unhe alag se explicitly request kiya ja sakta hai (ek separate method/attribute ke through) — lekin normal reading ke liye unki zaroorat nahi.

Comparison — Socket Approach Vs urllib Approach

Aspect	Socket (manual)	urllib
Lines of code	~10	~4
Connection banana	Manual (`connect()`)	Automatic
GET request banana/bhejna	Manual (string + encode + send)	Automatic
Headers handling	Khud parse karne parte (blank line dhoondhna)	Automatically separate/consumed
Data receive karna	Manual loop (`recv(512)`, 0-check)	Simple `for line in fhand`
Decode	Manual (zaroori)	Manual (zaroori — yahan bhi khud karna parta hai)

> **Key takeaway:** `urllib` network resource ko read karne ki complexity ko taqreeban **local file read karne jitna simple** bana deta hai — bas farq itna hai ke `import` alag hai, file open karne ka tareeqa alag hai, aur `decode()` manually karna parta hai.

Practical Example — Word Count Dictionary (File Jaisa Treatment)

```python
```

```python
import urllib.request

counts = dict()
fhand = urllib.request.urlopen('http://data.pr4e.org/romeo.txt')
for line in fhand:
    words = line.decode().split()
    for word in words:
        counts[word] = counts.get(word, 0) + 1
print(counts)
```

- Ye code URL se data read karke, har line ke words count karke, ek **dictionary** banata hai.
- **Important:** Ye code **bilkul waisa hi hai** jaisa hum ek **local file** ke sath likhte (dictionary banana, line-by-line loop, words pe inner loop) — **sirf 2 farq hain:**
  1. `import urllib.request` (file ke liye ye zaroori nahi hota)
  2. `urllib.request.urlopen(url)` se file open karna (local file `open()` ki jagah)
  3. `.decode()` call karna (kyunke line bytes hai, string nahi — local text file read karte waqt ye zaroorat nahi parti)
- Baaki poora logic (`.split()`, dictionary counting, etc.) **exactly wahi hai** jo local file processing mein hota.

HTML Web Pages Bhi Isi Tareeqe Se Read Ho Sakti Hain

- Jaisa text file (`.txt`) ke liye kiya, waisa hi HTML page ke liye bhi kiya ja sakta hai:

```python
fhand = urllib.request.urlopen('http://www.dr-chuck.com/page1.htm')
for line in fhand:
    print(line.decode().strip())
```

- Headers yahan bhi `urlopen()` ne "eat" kar liye — sirf raw HTML content (with `<`, `>` tags) output hota hai.
- Is se aage ek **web crawler** bhi likha ja sakta hai:
  - Page open karo → HTML mein se **links dhoondo** (parse karke) → un links ko bhi open karo → unke links dhoondo → aage badhte jao.
  - Yehi basic idea hai jis pe **search engine crawlers** (jese Google) kaam karte hain.
  - Fun fact/anecdote: Speaker guess karta hai ke Google ka **pehla web crawler bhi Python mein** likha gaya hoga, kyunke sirf itni si (4 lines) coding se web pages retrieve karke database mein daalna mumkin hai.

Aage Ka Challenge — HTML Parsing

- HTML data ko directly handle karna **messy/complex** hota hai:
  - Simple **regular expressions** ya `split()` se string parsing kaafi nahi hoti — HTML structure kaafi "crazy"/inconsistent ho sakta hai.
- Isliye agla topic hoga: ek **dedicated library** jo HTML parsing ko **asaan** bana degi (jese `BeautifulSoup` jesi libraries — course mein aage discuss hogi).

---

2. Active Recall Question Bank (Part 5)

1. `urllib` se web browser likhne mein roughly kitni lines lagti hain, aur socket approach mein kitni thi?
2. `urllib.request.urlopen()` internally kaunse steps automatically kar deta hai (list karo)?
3. `urlopen()` ka return value kis cheez ki tarah behave karta hai, aur is se kya practical fayda hota hai?
4. Kya `urllib` automatically bytes ko string mein decode kar deta hai? Sahi/Ghalat, wajah batao.
5. Basic `urlopen()` loop mein headers kyun nahi dikhte?
6. Socket approach aur urllib approach mein "headers handling" ka farq kya hai?

7. Word-count dictionary wale example mein, local file processing ke comparison mein kaunse 2-3 farq hain?
8. Kya HTML web pages ko bhi `urlopen()` se read kiya ja sakta hai? Kaise?
9. Web crawler ka basic idea kya hai (conceptually, is chapter ke context mein)?
10. HTML ko simple regex/split se parse karna kyun mushkil/inefficient hai?
11. Agla topic (agle section mein) kya discuss hoga, aur kyun zaroori hai?

---

3. Answer Key (Part 5)

1. `urllib` approach mein roughly **4 lines**, socket approach mein ~**10 lines** lagti thi.
2. Connection banana, GET request khud construct/encode karna, use bhejna, aur response ke **headers ko retrieve karke apne paas store** kar lena (bina unhe directly dikhaye).
3. Ye **file handle jaisa** behave karta hai — isi wajah se isay directly `for line in fhand:` jese for-loop mein use kiya ja sakta hai, bilkul jaise local file read karte hain.
4. **Ghalat.** `urllib` automatically decode nahi karta — line data **bytes** hi rehta hai, `.decode()` manually call karna zaroori hai.
5. Kyunke `urlopen()` headers ko internally "consume/eat" karke apne paas rakh leta hai — loop sirf actual content/data return karta hai, headers alag se dikhane ke liye explicit request karni parti hai.
6. Socket approach mein developer ko khud response mein **blank line dhoond kar** headers ko data se manually separate karna parta tha; urllib approach mein ye kaam **automatically** ho jata hai — headers already alag/consumed ho chuke hote hain.
7. Farq: (1) `import urllib.request` extra hota hai, (2) File `urllib.request.urlopen(url)` se "open" hoti hai (local `open()` ki jagah), (3) Har line pe `.decode()` call karna parta hai (kyunke line bytes hoti hai, jabke local text file read karne pe seedha string milti hai).
8. **Haan**, bilkul isi tareeqe se — `urlopen()` ko HTML page ka URL de kar, loop mein lines read karke `.decode()` kiya ja sakta hai; output raw HTML (tags samet) hota

hai kyunke headers already consume ho chuke hote hain.

9. Web crawler ka basic idea: ek page open karo, us mein se **links dhoondo/parse karo**, un links ko bhi open karo, unme se aage aur links nikalo — is tarah recursively pages retrieve karte jao (jese search engines websites ko index karte hain).
10. Kyunke HTML ka structure kaafi "messy/crazy" hota hai — simple regular expressions ya `.split()` string operations is complexity ko reliably handle nahi kar patay.
11. Agla topic: ek **dedicated HTML-parsing library** (jese BeautifulSoup) — isay isliye zaroori samjha gaya kyunke raw string methods se HTML parse karna unreliable/complex hai.

Part 6 — urllib Code Demo (`urllib1.py` aur `urlwords.py`)

1. Notes (Concept Explanation)

Ye section pichle (Part 5) concept ka hi **practical code walkthrough** hai — do sample files use hoti hain: `urllib1.py` aur `urlwords.py`.

`urllib1.py` — Basic urllib Recap

- Socket wale example ke comparison mein: jahan socket mein humein **import socket**, manual connection, GET command **encode** karke bhejna, aur data **manually read/write** karna parta tha — `urllib1.py` mein ye sab **gayab** ho jata hai.
- Bas ek **handle** banaya jata hai: `urllib.request.urlopen()` ko sirf ek **URL string** pass karke.
  - Koi manual encoding nahi.
  - Koi manual GET command banana nahi.
  - Lower-level `read()`/`write()` calls nahi — seedha ek simple **for-loop** se kaam ho jata hai.
- For-loop se jo `line` milti hai wo **bytes ki array** hoti hai — isliye:
  1. Pehle `.decode()` (bytes → string)

2. Phir `.strip()` (extra whitespace/newline hatana)

- Yani poora program **"super simple"** — halke se code mein pura HTTP GET + response handling ho jata hai.
- Note: Is basic version mein **headers nahi dikhte**, sirf actual content read hota hai. (Advanced use cases mein headers explicitly access kiye ja sakte hain — ye baad mein zyada complex applications mein dikhaya jayega.)

`urlwords.py` — Word Frequency Counting

- Ye essentially **wahi word-counting code** hai jo pehle local file ke sath likha gaya tha — is baar sirf source ek **URL** hai, local file nahi.
- **Sirf ek hi real farq hai:** jab tak `line.decode()` nahi kiya jata, tab tak wo "legitimate" Python string nahi ban ti — decode hone ke baad hi normal string operations (`.split()`, etc.) chal sakte hain.
- Baaki flow bilkul same hai:
  1. URL open karo (dictionary ke liye handle milta hai).
  2. Empty **dictionary** banao (word counts store karne ke liye).
  3. Har line pe loop chalao → `.decode()` karo → `.split()` se words nikalo.
  4. Har word pe inner loop chala kar count update karo (`counts[word] = counts.get(word, 0) + 1` jese pattern se).
- Run karne ka command jese: `python3 urlwords.py`
- Output ek **dictionary** hoti hai jisme har word ki frequency (kitni baar aaya) store hoti hai — is data pe aage sorting jese operations bhi kiye ja sakte hain.

Core Takeaway (Is Poore Demo Ka Essence)

> `urllib` ka sab se bara fayda ye hai ke ye **URLs ko Python ke andar files jaisa treat karna** possible bana deta hai. Sirf ek chhoti si cheez alag hai: shuru mein **`.decode()`** call karna zaroori hota hai (kyunke network se data bytes ki form mein aata hai) — iske baad sab kuch **bilkul waise hi** kaam karta hai jaise local file ke sath karte.

---

2. Active Recall Question Bank (Part 6)

1. `urllib1.py` mein socket wale approach ke comparison mein kya-kya "gayab" ho jata hai?
2. `urllib.request.urlopen()` ko kya pass kiya jata hai, aur is se kya milta hai?
3. `urllib1.py` mein data read karne ke liye lower-level `read()`/`write()` ki jagah kya use hota hai?
4. For-loop se milne wali `line` ka type kya hota hai, aur usay use karne se pehle kaunse do steps zaroori hain?
5. `urllib1.py` ke basic version mein headers dikhte hain ya nahi?
6. `urlwords.py`, pehle wale local-file word-count code se kis ek cheez mein differ karta hai?
7. `line.decode()` karne se pehle wo "legitimate Python string" kyun nahi hoti?
8. `urlwords.py` ka final output kya hota hai?
9. Is poore demo (Part 6) ka core/summary takeaway kya hai?

---

3. Answer Key (Part 6)

1. Manual socket import, manual connection banana, GET command ko manually encode karke bhejna, aur lower-level manual read/write handling — ye sab `urllib1.py` mein gayab ho jate hain.
2. Ek **URL string** pass ki jati hai; is se ek **handle** milta hai jisay for-loop mein use kiya ja sakta hai (file-handle jaisa behave karta hai).
3. Ek simple **for-loop** use hota hai, jo directly line-by-line data read kar leta hai.
4. `line` ka type **bytes (array of bytes)** hota hai; use karne se pehle (1) `.decode()` (bytes → string) aur (2) `.strip()` (extra whitespace/newline hatana) zaroori hai.
5. **Nahi** dikhte — basic version mein sirf actual content read hota hai, headers automatically consume ho jate hain (advanced use cases mein explicitly access kiye ja sakte hain).

6. Sirf ye farq hai ke `line.decode()` call karna zaroori hota hai, kyunke source ab local file ki bajaye ek **URL/network resource** hai — baaki poora logic (`.split()`, dictionary counting) bilkul same rehta hai.
7. Kyunke network se directly milne wala data **bytes** ki form mein hota hai, na ke Python ka native Unicode string — `.decode()` call karne ke baad hi wo ek asal/legitimate Python string banta hai jis pe string operations (`.split()`, etc.) reliably chal sakein.
8. Ek **dictionary** jisme har unique word ki **frequency (count)** store hoti hai — jise aage sort ya analyze kiya ja sakta hai.
9. `urllib` URLs ko Python ke andar **files jaisa treat karna** possible banata hai — sirf shuru mein `.decode()` ka ek extra step chahiye hota hai, uske baad sab kuch local file processing jaisa hi simple hota hai.

Part 7 — Web Scraping Aur Beautiful Soup

1. Notes (Concept Explanation)

Web Scraping / Web Spidering Kya Hai?

- **Web scraping (ya web spidering)** ka matlab hai:
    1. Ek web page **retrieve** karna
    2. Us page se **links extract** karna
    3. Un un-retrieved links ki ek **queue** banana
    4. Phir un pe **move** kar ke yehi process repeat karna
- Idea: agar kaafi time, energy, bandwidth, aur storage ho, to theoretically internet ke **zyadatar interconnected web pages** tak pohancha ja sakta hai (jo aapas mein links ke through connected hain).
- **Use cases:** apna blog data nikalna, kisi purani/retire ho rahi system se data nikalna, kisi cheez mein change hone pe notification bhejna (jese text message), apna khud ka **search engine** banana, waghera.

Ethical/Legal Caution — Har Website Scrape Nahi Hoti

- **Har website scraping allow nahi karti:**
  - Kuch websites **login required** karti hain, aur activity **track** karti hain — agar suspicious lage to **account band** kar sakti hain.
  - Kuch websites bina login ke bhi track karti hain aur agar zyada scraping detect ho to **IP address block** kar sakti hain.
- Isliye **caution zaroori hai:** kisi bhi site ko scrape karne se pehle uski policy (jese `robots.txt`, terms of service) padhni chahiye ke wo scraping allow karti hai ya nahi.

HTML Parsing Difficult Kyun Hai?

- Simple cases mein regular expressions ya `.split()` se kaam chal jata hai — lekin ye approach **scale nahi karta**:
  - Real-world web bohat sara **broken/malformed HTML** contain karta hai.
  - Browsers is broken HTML ko silently "fix" kar ke render kar dete hain (koi error nahi dikhate) — isliye developers ko pata hi nahi chalta ke unki site ka HTML technically ghalat hai, aur wo aise hi rehne deta hai.
  - Agar khud se custom parser likha jaye (regex/split se), to har naye "weird" tarike ke broken anchor tag pe code **crash/fail** ho sakta hai — aur aise weird variations **bohat zyada** hain.

Beautiful Soup — HTML Parsing Ka Solution

- **Beautiful Soup** ek library hai jo specifically HTML ke "messy" real-world nature ko handle karne ke liye banayi gayi hai.
- Isay kisi ne (developers ne) mahino tak kaam kar ke banaya — har tarah ki broken/nasty HTML situations ko already **compensate** kar diya gaya hai, aur ek **clean/simple interface** de diya gaya hai: bas **HTML do, tags wapis lo.**

Installation Ke Do Tareeqe

1. Agar computer configure kar sakte ho (apna system hai): Beautiful Soup ko **Python installer** ke through globally install kiya ja sakta hai (sab Python programs ke liye available).
2. Agar computer configure nahi kar sakte (jese school computer, USB stick pe kaam kar rahe ho): ek `BS4.zip` file download karke usay apni script (jese `urllinks.py`) ke sath ek folder (`bs4`) ki shakal mein rakha ja sakta hai — us folder se local import ho jayega.

Beautiful Soup Ka Basic Code Pattern

```python
from urllib.request import urlopen
from bs4 import BeautifulSoup

url = input('Enter - ')
html = urlopen(url).read()
soup = BeautifulSoup(html, 'html.parser')

tags = soup('a')
for tag in tags:
    print(tag.get('href', None))
```

Step-by-Step Breakdown

1. `from bs4 import BeautifulSoup` — Beautiful Soup library import karna.
2. User se **URL** input lena.
3. `urlopen(url).read()` — **Poora page ek sath (whole file) read** kiya jata hai — is baar loop nahi chalaya jata line-by-line, balke `.read()` se **saara content ek dafa mein** liya jata hai.
    - Ye theek hai jab tak file itni bari na ho ke memory issue ho.
4. Result **bytes** hota hai — lekin Beautiful Soup ko bytes aur UTF-8 dono ka pata hai, isliye developer ko manually decode karne ki zaroorat **nahi** parti —

`BeautifulSoup()` khud handle kar leta hai.

5. `BeautifulSoup(html, 'html.parser')` — bytes/HTML ko parse karke ek **Soup object** return karta hai.
   - Ye Soup object HTML ki **saari inconsistencies aur imperfections ko fix/handle** kar chuka hota hai internally.
   - Ab is object pe **queries** chalayi ja sakti hain.
6. `soup('a')` — Soup object ko **function ki tarah call** kar ke sirf **anchor tags (`<a href="...">`)** nikalna — anchor tags wo tags hain jo links represent karte hain.
7. `for tag in tags:` — Kyunke page mein **multiple anchor tags** ho sakte hain, loop se har tag process kiya jata hai.
8. `tag.get('href', None)` — Har anchor tag se uska `href` **attribute (actual link URL)** nikal kar print kiya jata hai.

- Example: agar `dr-chuck.com` pe ye run kiya jaye, to us page ka **ek external link** print ho jayega.

> Beautiful Soup ek **complex/feature-rich library** hai — upar wala code simple lagta hai, lekin poori library samajhne ke liye iski **documentation** dekhni parti hai (bohat sara aur bhi kiya ja sakta hai: sirf links nahi, images, text, specific tags by class/id, etc.)

---

2. Chapter 12 Ka Overall Summary (Recap)

Is poore chapter mein humne ye journey ki:

1. **TCP/IP Model** — Layered architecture samjha (link, internet, transport layers), aur **sockets** ka concept — do computers ke darmiyan ek "phone call" jaisa reliable connection.
2. **Application Protocols** — Ye rules decide karte hain ke socket connection pe **kya kaha jaye ga aur kis order mein** (jese HTTP).
3. **HTTP Protocol** — URL structure, request-response cycle, RFC/IETF standards, aur Telnet se manually HTTP request bhej kar dekha.

4. **Python Se Web Browser (Socket-Level)** — `socket` library use karke manually connect, GET request encode/send, aur data receive/decode kiya — poora browser sirf ~10 lines mein.
5. **Character Encoding** — ASCII, Unicode, UTF-8/16/32 ka farq, aur Python 3 mein strings (Unicode) vs bytes ka distinction, aur `encode()`/`decode()` ka sahi use.
6. `urllib` **Library** — Socket-level complexity ko abstract kar ke, URLs ko **local files jaisa** simple treat karna — sirf 4 lines mein web browser.
7. **Web Scraping Aur Beautiful Soup** — Real-world messy/broken HTML ko reliably parse karne ke liye ek dedicated library (**Beautiful Soup**) use karna, taake links (ya doosri cheezein) reliably extract ki ja sakein.

Core Theme Poore Chapter Ka

> Python complex, low-level networking concepts (TCP/IP, sockets, encoding, HTTP protocol) ko **chhote, simple programs** mein convert karna possible banata hai. Yehi wajah hai ke Python itna popular hai — ye **complexity ko simplicity mein** badal deta hai.

---

3. Active Recall Question Bank (Part 7)

1. Web scraping/spidering ka basic process kya hai (4 steps)?
2. Web scraping ke kuch practical use cases batao.
3. Websites scraping ko discourage/punish kaise kar sakti hain (2 tareeqe)?
4. Simple regex/split se HTML parse karna production-level pe kyun fail ho jata hai?
5. Broken HTML web pe kyun itni common hai (browsers ka role kya hai is mein)?
6. Beautiful Soup basically kya solve karti hai?
7. Beautiful Soup install karne ke do tareeqe kya hain?
8. `urlopen(url).read()` aur pehle wale line-by-line loop approach mein farq kya hai?
9. Beautiful Soup ko bytes decode karne ki zaroorat khud (manually) parti hai ya nahi?

10. `BeautifulSoup(html, 'html.parser')` call se kya milta hai, aur us object ki khaasiyat kya hai?
11. `soup('a')` kya return karta hai?
12. `tag.get('href', None)` ka kaam kya hai?
13. Chapter 12 mein overall journey kis order mein hui — TCP/IP se le kar Beautiful Soup tak, short mein batao.
14. Poore chapter ka "core theme" ek line mein kya hai?

---

4. Answer Key (Part 7)

1. (1) Web page retrieve karna, (2) Us se links extract karna, (3) Un-retrieved links ki queue banana, (4) Un pe move kar ke process repeat karna.
2. Apna blog data nikalna, retire ho rahi kisi system se data nikalna, kisi change pe notification/text bhejna, apna khud ka search engine banana.
3. (1) Login required kar ke activity track karna aur suspicious lagne pe **account band** karna, (2) Bina login ke bhi track kar ke suspicious activity pe **IP address block** karna.
4. Kyunke real-world web mein bohat sara **broken/malformed HTML** hota hai, aur is broken HTML ke itne alag-alag "weird" variations hote hain ke ek custom regex/split-based parser har naye case pe fail/crash ho jata hai — ye scale nahi karta.
5. Kyunke **browsers** broken HTML ko silently fix karke render kar dete hain, koi error message nahi dikhate — isliye developers ko apni site ki broken HTML ka pata hi nahi chalta, aur wo aise hi reh jati hai.
6. Beautiful Soup **HTML ki saari real-world imperfections/inconsistencies ko already handle/compensate** kar chuki hai — developer ko sirf HTML dena hota hai aur usay clean, query-able tags mil jate hain.
7. (1) Python installer ke through **globally install** karna (agar system configure kar sakte ho), (2) `BS4.zip` download kar ke apni script ke sath ek **local folder (bs4)** ki tarah rakhna (agar system configure nahi kar sakte, jese school computer/USB).

8. `urlopen(url).read()` **poora content ek sath (whole file)** read kar leta hai (loop nahi chalana parta), jabke pehle wala approach **line-by-line for-loop** se data process karta tha.
9. **Nahi**, manually decode karne ki zaroorat **nahi** parti — Beautiful Soup khud bytes aur UTF-8 ko samajh kar handle kar leta hai.
10. Ek **Soup object** milta hai, jo HTML ki saari inconsistencies fix kar chuka hota hai internally, aur is object pe **queries** (jese specific tags dhoondna) chalayi ja sakti hain.
11. `soup('a')` page ke saare **anchor (`<a>`) tags** (yani links) return karta hai.
12. Ye kisi anchor tag se uska `href` **attribute** (yani actual link URL) nikalta hai; agar attribute na mile to default value (`None`) return karta hai.
13. TCP/IP model aur sockets → application protocols → HTTP protocol aur URL/request-response cycle → socket-level Python web browser → character encoding (ASCII/Unicode/UTF-8) → `urllib` library (simple 4-line browser) → web scraping aur Beautiful Soup (reliable HTML parsing).
14. Python complex, low-level networking concepts ko **chhote aur simple programs** mein convert karna possible banata hai — complexity ko simplicity mein badal deta hai.

Part 8 — `urllinks.py` Live Demo (SSL Handling Ke Sath)

1. Notes (Concept Explanation)

Ye section Part 7 (Beautiful Soup) ka hi **live code demo/walkthrough** hai — file ka naam `urllinks.py` hai.

Setup — Beautiful Soup Install Karna (Recap)

- `urllinks.py` chalane se pehle **Beautiful Soup install** karna zaroori hai. Do tareeqe:
    1. **Global install:** Python's install process se, agar apna computer ho aur Beautiful Soup baar-baar use karni ho.

2. **Local/portable tareeqa (simpler):** `bs4.zip` download karo, usay **unzip** karke `urllinks.py` ke **sath usi folder** mein rakh do (subfolder ka naam `bs4`).

- Jab code mein `from bs4 import BeautifulSoup` likha jata hai, Python ya to (a) **global install location** se BeautifulSoup dhoondta hai, ya (b) agar wahan na mile to **local `bs4` folder** se import kar leta hai.

SSL Handling — HTTPS Sites Ke Liye Zaroori Hack

- Jin websites pe **SSL/HTTPS** hai, unhe access karne ke liye code mein ek chhota sa **"hack"** add karna parta hai — warna HTTPS sites access nahi hongi.
- Ye teen extra lines (SSL context banana, certificate verification ko bypass karna) zyada detail mein samajhne ki zaroorat nahi — bas itna yaad rakho ke:
  - Ye SSL **certificate verification errors ko ignore/bypass** karne ke liye hai.
  - Iska use karne ke liye ek `context` variable (jese `ctx`) banaya jata hai, aur `urlopen()` call mein ek extra parameter `context=ctx` pass kiya jata hai.
- **Important:** Agar ye lines na dali jayein, to `https://` wali sites access karte waqt error aa sakta hai — is se ye nahi pata chalta ke URL galat hai, balke masla ye hota hai ke us website ka SSL certificate Python ki "official trusted list" mein nahi hai.

Poora Flow (Recap + New Details)

1. User se ek **URL** input liya jata hai.
2. `urlopen(url).read()` — is baar **loop se line-by-line nahi**, balke **poori HTML ek sath, ek single bari string ki tarah** read ki jati hai (newlines included, jo har line ke end mein hote hain).
3. Ye result technically **Unicode string nahi** hai — ye **UTF-8 bytes** hai. Lekin **Beautiful Soup dono handle kar sakta hai** — UTF-8 bytes bhi aur Unicode strings bhi — is liye manually decode karne ki tension nahi.
4. `BeautifulSoup(html, 'html.parser')` — Beautiful Soup ko keh diya jata hai: "Ye messy/nasty HTML hai, isay parse karo aur samjho."
   - HTML **bohat flexible/inconsistent** hoti hai — jese line breaks kahin bhi ho sakte hain, tags ke around whitespace ka koi fix rule nahi — isliye simple

regex/split se parse karna practically mushkil hai.

5. Result: ek **"soup" object** milta hai (naam koi bhi rakh sakte ho, "soup" convention hai) — ye ek **clean proxy/representation** hota hai us messy HTML ka.

Beautiful Soup Ka Naam Kahan Se Aya?

- Library ka naam ek **children's story ke joke** se inspired hai.
- Idea: kisi ne pehle se hi wo **saari possible cheezein figure kar li hain** jo HTML parse karte waqt galat ho sakti hain — agar tum khud parser likho, to tumhe ye sab **dheere dheere khud discover** karna paray ga (trial and error se) — Beautiful Soup ye kaam already kar chuki hai.

Anchor Tags Extract Karna — Detail Mein

- **Anchor tag** wo tag hai jo `<a ...>` se shuru ho kar `</a>` pe khatam hota hai — is mein **attributes** (jese `href`), aur beech mein **text/link label** hota hai.
- `soup('a')` call karne se ek **list of tags** milti hai — kyunke ek page mein **bohat saare** anchor tags ho sakte hain (example: `dr-chuck.com` pe **45 anchor tags** hain).
- Phir ek **loop** se in sab tags ke through "hop" kiya jata hai (ek tag se doosre tag), aur har tag se uska **`href` attribute value** nikala jata hai.

Beautiful Soup Ki Real Power — Broken HTML Ko Bhi Handle Karna

- Demo mein ek real example dikhaya gaya: kisi ek anchor tag mein **closing quote missing** thi (`href="...` ki closing `"` nahi thi) — ye technically **invalid/broken HTML** hai.
- Phir bhi Beautiful Soup ne isay **sahi tarah handle kar liya** aur link nikal diya — ye exactly wo cheez hai jo Beautiful Soup ko itna valuable banati hai: **real-world ki chhoti-moti HTML mistakes ko silently sambhal lena.**

Live Testing — Command Line Se Run Karna

Command: `python3 urllinks.py`

1. **Simple test:** `http://www.dr-chuck.com/page1.htm` → sirf **1 link** milta hai (kyunke us specific page pe sirf ek hi anchor tag hai).
2. **Complex test:** `http://www.dr-chuck.com/` → **bohat saare links** milte hain (homepage pe zyada anchor tags hain) — demo mein dikhaya gaya ke sab links successfully nikal aate hain, chahe unme koi chhoti "gadbad"/inconsistency bhi ho.
3. **HTTPS test:** `https://www.si.umich.edu/` → Ye **SSL/HTTPS site** hai, is liye SSL-handling wale extra code (context hack) ke bina ye request **fail** ho jati; us hack ke sath ye successfully bohat saare links retrieve kar leta hai.
    - Clarify: SSL error ka matlab ye **nahi** ke URL galat hai — matlab ye hai ke us website ka **certificate Python ki trusted certificate list mein officially registered nahi** hai.

Overall Summary (Is Demo Ka)

> Ye demo dikhata hai ke `urllib` (URL open/read) aur `BeautifulSoup` (HTML parse) ko **milaake** kaise real-world web pages (chahe wo thoda broken HTML rakhte hon, ya HTTPS pe hon) se **reliably links extract** kiye ja sakte hain — is sab ke liye sirf ek chhota sa Python script kaafi hai.

---

2. Active Recall Question Bank (Part 8)

1. `urllinks.py` chalane se pehle kya install karna zaroori hai, aur do tareeqe kya hain?
2. `bs4.zip` wale (local) tareeqe mein file structure kaisa hota hai?
3. `from bs4 import BeautifulSoup` line kaam kaise karti hai (do possible sources)?
4. SSL/HTTPS wali sites access karne ke liye code mein kya extra add karna parta hai?
5. Agar SSL-handling code na dala jaye to HTTPS site access karte waqt kya hoga, aur iska matlab kya hai (URL galat hai ya kuch aur)?
6. Is demo mein poori HTML kaise read ki jati hai — line-by-line ya ek sath?
7. `urlopen().read()` se jo data milta hai, uska "type" kya hai (Unicode string ya kuch aur), aur Beautiful Soup ye kaise handle karta hai?

8. HTML parsing regex/split se mushkil kyun hai — is demo mein diya gaya specific reason kya tha?
9. "Beautiful Soup" naam kahan se inspired hai, aur is naam ka symbolic matlab kya hai (kis idea ko represent karta hai)?
10. Anchor tag kya hota hai — teen parts batao.
11. `soup('a')` kya return karta hai, aur `dr-chuck.com` pe kitne anchor tags mile the demo mein?
12. Demo mein Beautiful Soup ne kaunsi "broken HTML" situation ko successfully handle kiya?
13. `page1.htm` aur `dr-chuck.com/` (homepage) pe run karne se links ki count mein farq kyun tha?
14. `https://www.si.umich.edu/` test karte waqt SSL hack ke bina kya hota, aur uske sath kya hua?

---

3. Answer Key (Part 8)

1. **Beautiful Soup** install karna zaroori hai. Do tareeqe: (1) Python installer se **globally install** karna, (2) `bs4.zip` download-unzip karke script ke **sath local folder** ki tarah rakhna.
2. `urllinks.py` file ke **sath usi folder mein** ek subfolder `bs4` hota hai (jo `bs4.zip` ko unzip karne se milta hai), jisme Beautiful Soup ka code hota hai.
3. Ye line ya to Python ke **global install location** se BeautifulSoup import karti hai, ya agar wahan na mile to **local `bs4` folder** se import kar leti hai.
4. Ek **SSL context** (jese `ctx`) banaya jata hai jo certificate verification errors ko ignore/bypass karta hai, aur `urlopen()` call mein `context=ctx` parameter pass kiya jata hai.
5. HTTPS site access karte waqt **error/failure** aayega. Iska matlab **URL galat nahi hai** — balke us website ka **SSL certificate Python ki officially trusted certificate list mein registered nahi** hai.

6. **Ek sath** — `urlopen(url).read()` se **poori HTML ek single string** ki tarah mil jati hai (line-by-line loop nahi chalaya jata).
7. Ye data **UTF-8 bytes** hota hai (Unicode string nahi); Beautiful Soup dono (UTF-8 bytes aur Unicode strings) ko handle karna janta hai, isliye manually decode karne ki zaroorat nahi parti.
8. Kyunke HTML bohat **flexible/inconsistent** hoti hai — jese line breaks kahin bhi aa sakte hain, tags ke structure mein variations ho sakte hain — is wajah se fixed pattern wali regex/split reliably kaam nahi karti.
9. Ek **children's story ke joke** se — symbolic taur pe ye represent karta hai ke kisi ne pehle se hi HTML ki saari "messy/nasty" possible situations already figure kar li hain aur unhe compensate kar diya hai, taake developer ko khud trial-and-error se ye sab discover na karna paray.
10. (1) **Attributes** (jese `href`), (2) **Tag khud** (`<a>` se `</a>` tak), (3) Tag ke andar ka **text/label**.
11. `soup('a')` page ke **saare anchor tags ki ek list** return karta hai; demo mein `dr-chuck.com` pe **45 anchor tags** mile the.
12. Ek anchor tag mein `href` **attribute ki closing quote missing** thi (invalid/broken HTML) — phir bhi Beautiful Soup ne usay sahi tarah parse kar ke link nikal liya.
13. Kyunke `page1.htm` ek specific chhota page tha jisme **sirf ek anchor tag** tha, jabke `dr-chuck.com/` (homepage) mein **bohat zyada links** the — isliye output count mein farq tha.
14. SSL hack ke bina request **fail** ho jati (certificate error ki wajah se); SSL context (`ctx`) hack add karne ke baad request successful hui aur **bohat saare links** retrieve ho gaye.